

Terminal Mini-Games Hub

Project Report
Architecture, features, data storage and code review

Language	Python 3 (standard library only)
Entry point	python main.py
Modules	main.py, games.py, data_manager.py
Storage	score_history.json (local JSON file)
Total size	295 lines of Python across 3 files, plus README and design notes

1. Executive Summary

Terminal Mini-Games Hub is a small command-line application that bundles five text-based games behind a single menu. A player picks a game, plays it, and if they win, the score is written to a local JSON file. A sixth menu option shows the ten most recent saved scores.

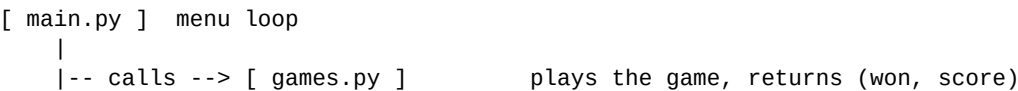
The project is cleanly organised into three modules with clear responsibilities: an interface layer, a game-logic layer and a storage layer. It has no third-party dependencies and runs anywhere Python 3 is installed. The review in section 7 found no crashes in normal play, but several small bugs and design gaps, the most notable being that invalid input in *Guess the Number* wastes an attempt and that the "High Scores" screen actually shows recent history.

2. Architecture

The program follows a simple three-module layout. `main.py` is the only module the user runs, and it imports the other two.

Module	Lines	Responsibility
main.py	62	Interface layer. Prints the menu, reads the choice, calls the chosen game and passes winning results to the storage layer.
games.py	179	Game logic. Five functions, one per game, each running its own input loop and returning a tuple (won, score).
data_manager.py	54	Storage. Creates the JSON file, appends new score records and prints the last ten entries.

Execution and data flow:



```

|
|-- calls --> [ data_manager.py ] init_db(), save_score(), show_scores()
|
|
v
[ score_history.json ]

```

Program flow. On start, `main()` calls `init_db()` to make sure the JSON file exists. It then loops: show menu, read choice, dispatch, wait for Enter. Options 1 to 5 run a game and call `save_score()` only when the game returns `won = True`. Option 6 prints the history and option 7 exits.

3. Games and Scoring

Game	How it works	Score on win
Guess the Number	Secret number 1-100, seven attempts, with too-high / too-low hints.	(7 - attempt + 1) x 10. Ranges from 70 (first try) to 10 (seventh try).
Word Guess	Random word from a list of six. Guess one letter at a time with six lives; wrong letters cost a life.	Remaining lives x 15. Ranges from 15 to 90.
Rock, Paper, Scissors	Best-of-three against a random computer. Ties replay the round.	Flat 30.
Tic-Tac-Toe	Player is X and moves first. The computer (O) picks a random open slot. Slots are numbered 1-9.	Flat 50 for a win. Draws and losses score 0.
Higher or Lower	A number 1-100 is shown; guess whether the next random number is higher or lower. Five correct in a row wins; one wrong guess ends the game.	Flat 40.

All five functions share the same contract: they return a `(bool, int)` pair, which keeps `main.py` simple and makes it easy to add a sixth game.

4. Data Storage

Scores are stored in `score_history.json` in the current working directory as a JSON array of records. Each record has three fields:

Field	Type	Example
game	string	"Tic-Tac-Toe"
score	integer	50
timestamp	string, YYYY-MM-DD HH:MM:SS	"2026-09-28 20:39:19"

Current data. The supplied file contains a single record: a Tic-Tac-Toe win worth 50 points, saved on 2026-09-28 at 20:39:19.

Functions in `data_manager.py`:

- `init_db()` - creates an empty array if the file does not exist.
- `save_score(game, score)` - loads the array, appends a timestamped record, writes it back with indentation and prints a confirmation.
- `show_scores()` - prints the last ten records, newest first, in a formatted table.

5. Documentation Review

The project ships with a README (user-facing overview) and a design document (`statement.md`) describing structure, data format and scoring. Both match the code closely: the module roles, the five games, the scoring values and the JSON schema are all accurate. Small issues:

- The README's final code block (`python main.py`) is never closed with a closing triple backtick, so it may render incorrectly.
- The design file name is spelled `statement.md` (missing an "e").
- The design document says "player history", but the program has no concept of players. Scores are not tied to a name.
- The design document says Guess the Number points scale with "unused attempts remaining". The code actually awards unused attempts plus one, times ten.

6. Strengths

- Clear separation of interface, logic and storage.
- Zero dependencies and a very small footprint.
- A consistent (`won`, `score`) return contract across games.
- Input validation in every game, so bad input does not crash the program.
- Corrupt or missing JSON is handled without a traceback.
- Documentation is present and largely accurate.

7. Code Review Findings

Ordered roughly by impact. Line references are to the supplied files.

#	Area	Finding	Suggested fix
1	Guess the Number	The attempt counter is a <code>for</code> loop. When the user enters non-numeric text, <code>continue</code> still advances the loop, so a typo silently costs one of the seven tries.	Use a <code>while</code> loop with a manual counter that only increments on valid guesses.
2	<code>data_manager.save_score</code>	If the JSON file is corrupt, the error is caught and data is reset to an empty list. The next write then overwrites the whole file, permanently losing any recoverable history.	Back up the bad file (for example rename to <code>.bak</code>) before starting a fresh list.
3	Menu option 6	The menu says "High Scores" but <code>show_scores()</code> prints the ten most recent entries in time order, not the highest scores.	Either rename it "Recent Scores" or sort by score for a true leaderboard.
4	Tic-Tac-Toe input	Entering 0 or a negative number produces a negative index, which Python accepts. If that slot is free the loop simply re-prompts with no message, and if it is taken the message says "Slot taken" for a slot the user never chose.	Check <code>1 <= choice <= 9</code> before indexing.
5	Higher or Lower	The message "Streak reset" is misleading, since one wrong answer ends the game. Also, when the starting number is near 1 or 100 the correct answer is almost certain, which weakens the game.	Reword the message, or make a wrong guess reset the streak and continue. Optionally start from 20-80.
6	Persistence	Only wins are saved, and scores are not linked to a player name, so the history cannot show attempts, losses or who played.	Optionally add a player name and a result field.

#	Area	Finding	Suggested fix
7	File path	DB_FILE is a relative path, so running the program from a different folder creates or reads a different score file.	Build the path from <code>__file__</code> .
8	Robustness	Ctrl+C or end-of-input at any prompt raises an uncaught exception and prints a traceback. Writes to the JSON file are not atomic.	Catch <code>KeyboardInterrupt</code> and <code>EEOFError</code> in <code>main()</code> ; write to a temp file and rename.
9	Maintainability	The five-branch <code>if/elif</code> chain in <code>main()</code> repeats the same pattern. Game names appear both in the menu and in the save calls.	Use a dictionary mapping menu number to (name, function).
10	Gameplay	The Tic-Tac-Toe computer plays randomly and always moves second, so the 50-point award is easy to earn. Word Guess uses only six fixed words.	Add a blocking/winning-move heuristic and a larger word list.

8. Testing Notes

No automated tests are included. The game functions read from `input()` and use `random` directly, so they are best tested by patching `builtins.input` and seeding `random`. The scoring formulas and the storage functions (using a temporary file path) are simple to cover with unit tests. Suggested first cases: a full winning path for each game, invalid input in each prompt, corrupt JSON on save, and the ten-entry cut-off in `show_scores()`.

9. Conclusion

Terminal Mini-Games Hub is a tidy, readable project that does what its documentation says. The architecture is sound and easy to extend. Fixing the attempt-counting bug in Guess the Number, protecting the score file from being overwritten after corruption and renaming or reworking the high-score screen would remove the most noticeable rough edges. Adding tests and player names would be natural next steps.